A7 Analysis – 6012 – Jon Pulsipher

1) Explain the hashing function you used for BadHashFunctor. Be sure to discuss why you expected it to perform badly (i.e., result in many collisions).

BadHashFunctor returns the int value of the first character in a string. Any strings starting with the first character will collide – ‘cat’ and ‘creeper’, for example.

2) Explain the hashing function you used for MediocreHashFunctor. Be sure to discuss why you expected it to perform moderately (i.e., result in some collisions).

MediocreHashFunctor adds up the int value of all characters in a string. Collisions will be rare, but any strings with the same characters will collide – ‘cat’ and ‘act’, for example.

3) Explain the hashing function you used for GoodHashFunctor. Be sure to discuss why you expected it to perform well (i.e., result in few or no collisions).

GoodHashFactor uses the Fowler-Noll-Vo hash I pulled from some quick internet research. It starts with a big prime number, then xor’s it with each character and multiplies it with a big prime number. Why do I expect this to work? Well, because it’s a named hashing solution that was high in the search results. I believe there’s something about prime numbers that helps guarantee these hashes are more likely to be unique.

4) Design and conduct an experiment to assess the quality and efficiency of each of your three hash functions. Briefly explain the design of your experiment. Plot the results of your experiment. Since the organization of your plot(s) is not specified here, the labels and titles of your plot(s), as well as, your interpretation of the plots is important. A recommendation for this experiment is to create two plots: one that shows the number of collisions incurred by each hash function for a variety of hash table sizes, and one that shows the actual running time required by various operations using each hash function for a variety of hash table sizes.

Using a 55k-word dictionary, I will record the running time and number of collisions for adding that dictionary into a hash table. I will temporarily modify the hashTable to return true when a collision occurs.

	BadHashFunctor	MediocreHashFunctor	GoodHashFunctor
Collisions	45351	45351	45351
t for 45402 word dictionary	104ms	104ms	106ms

5) What is the cost of each of your three hash functions (in Big-O notation)? Note that the problem size (N) for your hash functions is the length of the String, and has nothing to do with the hash table itself. Did each of your hash functions perform as you expected (i.e., do they result in the expected number of collisions)?

BadHashFactor – O(1), as it just checks the first character.

MediocreHashFactor – O(N), as it must check all characters.
GoodHashFactor – O(N), as it much check all characters.

My hash functions all seemed to perform equally bad. The performance for badHashFunctor does seem to match what one would expect from only distinguishing between the 1st character. I do not know why the other two hashFunctors weren't better. It is likely an issue in my test if not my implementation.